COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S3
DATE PERFORMED:	7/28/2026	**DATE SUBMITTED:**	8/01/2026
NAME:	Dimaguiba,Lorenzo James M.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

SEATWORK NO. 4.1 Array-Based vs STL Stack Implementations

1. PROCEDURE OUTPUT

ILO A:

CPE_Dimaguiba > HOA4P1 > Stacks > Activity > Dimaguiba_Stacks_STL.cpp > main()

```cpp
#include <iostream>
#include <stack> //call the stack from STL

int main(){

    std::stack <int> newStack; //created an onject
    std::stack<int> newerStack;
    //test if stack is empty
    std::cout<<"Is the stack empty? "<<newStack.empty()<<std::endl;

    //insert 3 element
    newStack.push(3);
    std::cout<<"Top of stack: "<< newStack.top()<<std::endl;
    newStack.push(2);
    std::cout<<"Top of stack: "<< newStack.top()<<std::endl;
    newStack.push(1);
    std::cout<<"Top of stack: "<< newStack.top()<<std::endl;

    //test if stack is empty
    std::cout<<"Is the stack empty? "<<newStack.empty()<<std::endl;

    std::cout<<"Size of the stack is: "<<newStack.size()<<std::endl;

    newStack.push(6);
    std::cout<<"Top of stack: "<< newStack.top()<<std::endl;
    newStack.push(7);
    std::cout<<"Top of stack: "<< newStack.top()<<std::endl;
    newStack.push(8);
    std::cout<<"Top of stack: "<< newStack.top()<<std::endl;

    std::cout<<"Size of the stack is: "<<newStack.size()<<std::endl;

    //make second stack
    newerStack.push(9);
    std::cout<<"Top of stack: "<< newerStack.top()<<std::endl;
    newerStack.push(10);
    std::cout<<"Top of stack: "<< newerStack.top()<<std::endl;
    newerStack.push(11);
    std::cout<<"Top of stack: "<< newerStack.top()<<std::endl;
    newerStack.push(12);
    std::cout<<"Top of stack: "<< newerStack.top()<<std::endl;
    newerStack.push(13);
    std::cout<<"Top of stack: "<< newerStack.top()<<std::endl;
    newerStack.push(14);
    std::cout<<"Top of stack: "<< newerStack.top()<<std::endl;

    // swap the content of public member functions
    newStack.swap(newerStack);

    //emplace
    newStack.emplace(67);
    std::cout<<"Top of stack: "<< newStack.top()<<std::endl;

    //popping the stackD
    while (!newStack.empty()){
        std::cout<<newStack.top()<<" \n";
        newStack.pop();
    }

    return 0;

}
```

Output:

```
Is the stack empty? 1
Top of stack: 3
Top of stack: 2
Top of stack: 1
Is the stack empty? 0
Size of the stack is: 3
Top of stack: 6
Top of stack: 7
Top of stack: 8
Size of the stack is: 6
Top of stack: 9
Top of stack: 10
Top of stack: 11
Top of stack: 12
Top of stack: 13
Top of stack: 14
Top of stack: 67
67
14
13
12
11
10
9
```

Observation/Analysis:

It uses the STL stack library to do operations. Different operations call for different names. In a stack, the user may push and pop. Stack only enters and exits elements from the TOP of the list. Pushing puts the element on top, while popping removes the element on top as seen here. The top of the stack may be seen using the "stack.top()" function, and to check whether something is empty or not, use (.empty()). Swapping is also possible to swap between 2 different stacks. Emplace works similarly to push but creates the object before it exists.

ILO B:

CPE_Dimaguiba > HOA4P1 > Stacks > ILO A > main.cpp > main()

```cpp
#include <iostream>

const size_t maxCap = 100;
int stack[maxCap];
int top = -1, newData, elements;

//prototyping
bool isFull();
bool isEmpty();
void push();
void pop();
void Top();

//display all
void displayAll();

int main(){
    std::cout<<"Enter the number of maximum elements for the new stack (max: 100): ";
    std::cin>>elements;

    while(true){
        int choice = 0;
        std::cout<<"Select Stack Operation:"<<std::endl;
        std::cout<<"1. PUSH\n2. POP\n3. VIEW TOP\n4. ISEMPTY\n5. ISFULL\n6.DISPLAYALL"<<std::endl;
        std::cin >> choice;

        switch(choice){
            case 1:
                push();
                break;

            case 2:
                pop();
                break;

            case 3:
                Top();
                break;

            case 4:
                std::cout<<isEmpty()<<std::endl;
                break;

            case 5:
                std::cout<<isFull()<<std::endl;
                break;

            case 6:
                std::cout<<"The current stack: ";
                displayAll();
                break;

            default:
                std::cout<<"Invalid Choice"<<std::endl;
                break;
        }
    }
```

```cpp
    return 0;
}

bool isEmpty(){
    //top is -1
    if(top == -1){
        return true;
    }
    return false;
}

bool isFull(){
    //not equal to maxCap
    if(top >= elements - 1){
        std::cout<<"Stack Overflow"<<std::endl;
        return true;
    }
    return false;

}

void push(){
    //check if stack is full, if yes, return error
    if (isFull()){
        return;
    }
    //ask user input values to be pushed into stack
    std::cout<<"What number would you like to add? ";
    std::cin>>newData;
    //store inside stack
    stack[++top] = newData;
}

void pop(){
    // check if empty
    if (isEmpty()){
        return;
    }

    //display element to be popped
    std::cout<<"Popping: "<<stack[top]<<std::endl;

    //decrement top:
    top--;
}

void Top(){
    //error checking
    if(isEmpty()){
        std::cout<<"Stack Underflow"<<std::endl;
        return;
    }
    //display top
    std::cout<<"The top element is: "<<stack[top]<<std::endl;

}

void displayAll(){
    if (isEmpty()){
        std::cout<<"Nothing to display!"<<std::endl;
        return;
    }
    for(int i = top; i >= 0; i--){
        std::cout<<stack[i]<<" ";
    }
    std::cout<<std::endl;
}
```

Output:

```
Enter the number of maximum elements for the new stack (max: 100): 2
```

Menu:

```
Select Stack Operation:
1. PUSH
2. POP
3. VIEW TOP
4. ISEMPTY
5. ISFULL
6.DISPLAYALL
```

Push:

```
1
What number would you like to add? 6
```

```
What number would you like to add? 7
```

View All:

```
6.DISPLAYALL
6
The current stack: 7 6
```

Full or empty:
Returns 0 or 1:

```
1
```

true or false

```
Stack Overflow
```

```
Stack Underflow
```

Observation:
Works exactly as before, only this time, done through an array implementation. Done through functions, since done with arrays, one must specify the max capacity of the array stack. The number of elements however is decided by the user. A menu is also seen. Pushing works differently in that it not only assigns an element into the array, it moves the current position of the top element forward. Same with pop but backwards. They both have error checkers if the stack is full or empty. The isFull and isEmpty works to return true or false (1/0) if it is full or empty. To display all the elements in the stack, it runs a loop equal to the top value till all is printed.

ILO B2:
Main.cpp

CPE_Dimaguiba > HOA4P1 > Stacks > ILOB2 > main.cpp > main()

```cpp
#include <iostream>
#include "listStack.h"

int main(){

    std::cout<<"------Pushing 'I' 'E' 'E' 'E'------ \n";

    //pushing
    push('I');
    push('E');
    push('E');
    push('E');

    //display top
    Top<char>();

    //display all
    displayAll<char>();

    //pop and print each element
    std::cout<<"-----popping 'E' 'E' 'E' 'I'----- \n";
    while (!isEmpty<char>()){
        std::cout<<"popping: "<<pop<char>()<<std::endl;
    }

    //test stack underflow
    pop<char>();


    return 0;
}
```

Header File:

CPE_Dimaguiba > HOA4P1 > Stacks > ILO82 > C listStack.h > ...

```
#ifndef LISTSTACK_H
#define LISTSTACK_H

//class node with data and next members
template <typename T>
class Node{
    public:
        T data;
        Node<T>* next = nullptr;
};

//create a pointer/reference tip that points to current head

//global
template <typename T>
Node<T>* head = nullptr;

//pushing
template <typename T>
void push(T newData){
    //create new node
    Node<T>* newNode = new Node<T>;
    //assign data to the new node
    newNode->data = newData;
    //point the newNode to current top
    newNode->next = head<T>;
    //assign the new top to the Node
    head<T> = newNode;
}

//is empty
template <typename T>
bool isEmpty(){
    return head<T> == nullptr;
}

template <typename T>
T pop(){
    //if stack is empty
    if (isEmpty<T>()){
        std::cout<<"Stack underflow!"<<std::endl;
    }

    //saving current top node to a temporary variable
    Node<T>* temp =  head<T>;

    //store the data to a temp variable
    T tempVal = temp->data;

    //move head to next node
    head<T> = head<T>->next;
    //free memory
    delete temp;
    //return temporary variable data
    return tempVal;
}

template <typename T>
void Top(){
    //what if the stack is empty
    if(isEmpty<T>()){
        std::cout<<"Stack Underflow!"<<std::endl;
        return;
    }
    std::cout<<"Current Top:"<<head<T>->data<<std::endl;
}
```

```
template <typename T>
void displayAll(){
    if (isEmpty<T>()){
        std::cout<<"Stack Underflow!"<<std::endl;
        return;
    }
    Node<T>* current = head<T>;
    std::cout<<"Current Stack: ";
    while (current != nullptr){
        std::cout<<current->data<<" ";
        current = current->next;
    }
     std::cout<<std::endl;
}


#endif
```

Output:

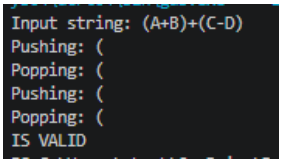
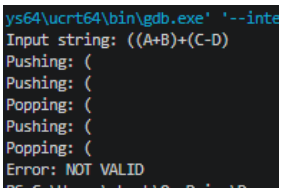
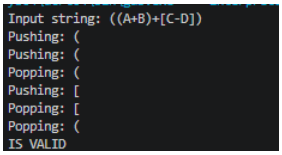
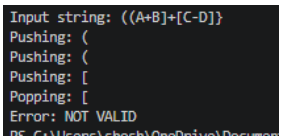
```
------Pushing 'I' 'E' 'E' 'E'------
Current Top:E
Current Stack: E E E I
-----popping 'E' 'E' 'E' 'I'-----
popping: E
popping: E
popping: E
popping: I
Stack underflow!
```

Observation:

It works as simple as array-based implementations but with the added benefit of being able to actually free memory and allow deletion of elements on the stack. Push and pop work similarly in that pushing, adds a node, while pop removes and displays the node removed. The need for an isFull() function wasn't used as the only limit to the stack is your system's memory. Top and Empty functions check the head mainly if it is empty or occupied; returns a boolean answer. If it is empty the program will let the user know.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C:

Expression	Valid (Y/N)	Output (Console Screenshot)	Analysis
(A+B)+(C-D)	Y		Whenever it encounters an opening symbol, it pushes it into the stack. Whenever it finds a matching closing symbol, it pops one of the opening symbols as it finds its pair. This goes for the rest too.
((A+B)+(C-D)	N		Does the same thing as before, only this time, it triggers the empty stack error as one parenthesis did not find itself a pair. With 3 opening parentheses, with only 2 pairs, there is one left behind.
((A+B)+[C-D])	Y		The checker for the match also includes other opening and closing symbols including brackets and braces. It follows the same concepts and code finding matches and the like.
((A+B]+[C-D]}	N		One symbol had no pair. Making it not valid. The problem is one of the brackets was missing an opening pair after popping, leaving an empty stack. Hence, its validity.

CODE:
Stack using Arrays

VSCODES > Stax > ILO C > ARR > mainArr.cpp > main()

```cpp
#include <iostream>
#include <string>

const size_t maxCap = 100;
int top = -1;
char stack[maxCap];
char newData;
std::string checkword;
int choice, elements;
//prototyping
bool isFull();
bool isEmpty();
void push(char newData);
void pop();
void Top();


int main(){
    std::cout<<"Input string: ";
    std::cin>>checkword;
    elements = checkword.length();
    for (int i = 0; i < checkword.length(); i++){
        if (checkword[i] == '(' or checkword[i] == '[' or checkword[i] == '{' ){
            std::cout<<"Pushing: "<<checkword[i]<<std::endl;
            push(checkword[i]);
        }
        else if (checkword[i] == ')' || checkword[i] == '}' || checkword[i] == ']' ){
            if(isEmpty()){
                std::cout<<"ERROR! STACK EMPTY!"<<std::endl;
                return 0;
            }

            if(stack[top] == '(' && checkword[i] == ')' || stack[top] == '{' && checkword[i]=='}' || stack[top] == '[' && checkword[i] == ']' ){
                pop();
            }

            }
    }
    if(!isEmpty()){
        std::cout<<"Error: NOT VALID"<<std::endl;
    }
    else{
        std::cout<<"IS VALID"<<std::endl;
    }
    return 0;
}


bool isEmpty(){
    if (top == -1){
        return true;
    }
    return false;
}

bool isFull(){
    //not equal to maxCap
    if(top >= elements - 1){
        std::cout<<"Stack Overflow"<<std::endl;
        return true;
    }
    return false;
}

void push(char newData){
    //check if stack is full, if yes, return error
    if (isFull()){
        return;
    }
```

```
70      //store inside stack
71      stack[++top] = newData;
72  }
73
74  void pop(){
75      // check if empty
76      if (isEmpty()){
77          return;
78      }
79
80      //display element to be popped
81      std::cout<<"Popping: "<<stack[top]<<std::endl;
82
83      //decrement top:
84      top--;
85  }
86
87  void Top(){
88      //error checking
89      if(isEmpty()){
90          std::cout<<"Stack Underflow"<<std::endl;
91          return;
92      }
93      //display top
94      std::cout<<"The top element is: "<<stack[top]<<std::endl;
95
96  }
97
98  void displayAll(){
99      if (isEmpty()){
100         std::cout<<"Nothing to display!"<<std::endl;
101         return;
102     }
103     for(int i = top; i >= 0; i--){
104         std::cout<<stack[i]<<" ";
105     }
106     std::cout<<std::endl;
107 }
```

Stack Using Linked Lists:

VSCODES > Stax > ILO C > LL > mainLL.cpp > ...

```cpp
#include <iostream>
#include <string>
#include "listStack.h"

std::string checkword;
/*
1. Create a Stack.
2. While(end of input is not reached) {
a) If the character read is not a symbol to be balanced, ignore it.
b) If the character is an opening symbol, push it onto the stack.
c) If it is a closing symbol:
i) Report an error if the stack is empty.
ii) Otherwise, pop the stack.
d) If the symbol popped is not the corresponding opening symbol, report an error.
3. At the end of input, if the stack is not empty: report an error.*/

int main(){
    std::cout<<"Input string: ";
    std::cin>>checkword;
    for (int i = 0; i < checkword.length(); i++){
        if (checkword[i] == '(' or checkword[i] == '[' or checkword[i] == '{' ){
            std::cout<<"Pushing: "<<checkword[i]<<std::endl;
            push(checkword[i]);
        }
        else if (checkword[i] == ')' || checkword[i] == '}' || checkword[i] == ']' ){
            if(isEmpty<char>()){
                std::cout<<"ERROR! STACK EMPTY!"<<std::endl;
                return 0;
            }

            char top = head<char>->data;
            if(top == '(' && checkword[i] == ')' || top == '{' && checkword[i]=='}' || top == '[' && checkword[i] == ']' ){
             std::cout<<"Popping: "<<top;
                pop<char>();
                std::cout<<std::endl;
        }

        }
    }
    if(!isEmpty<char>()){
        std::cout<<"Error: NOT VALID"<<std::endl;
    }
    else{
        std::cout<<"IS VALID"<<std::endl;
    }
    return 0;
}
```

Linked List HEADER File:

VSCODES > Stax > ILO C > LL > C listStack.h > ...

```
#ifndef LISTSTACK_H
#define LISTSTACK_H

//class node with data and next members
template <typename T>
class Node{
    public:
        T data;
        Node<T>* next = nullptr;
};

//create a pointer/reference tip that points to current head

//global
template <typename T>
Node<T>* head = nullptr;

//pushing
template <typename T>
void push(T newData){
    std::cout<<"Pushing: "<<newData<<std::endl;
    //create new node
    Node<T>* newNode = new Node<T>;
    //assign data to the new node
    newNode->data = newData;
    //point the newNode to current top
    newNode->next = head<T>;
    //assign the new top to the Node
    head<T> = newNode;
}

//is empty
template <typename T>
bool isEmpty(){
    return head<T> == nullptr;
}

template <typename T>
T pop(){
    //if stack is empty
    if (isEmpty<T>()){
        std::cout<<"Stack underflow!"<<std::endl;
    }

    //saving current top node to a temporary variable
    Node<T>* temp =  head<T>;

    //store the data to a temp variable
    T tempVal = temp->data;

    //move head to next node
    head<T> = head<T>->next;
    //free memory
    delete temp;
    //return temporary variable data
    return tempVal;
}

```

```cpp
template <typename T>
void Top(){
    //what if the stack is empty
    if(isEmpty<T>()){
        std::cout<<"Stack Underflow!"<<std::endl;
        return;
    }
    std::cout<<"Current Top:"<<head<T>->data<<std::endl;
}

template <typename T>
void displayAll(){
    if (isEmpty<T>()){
        std::cout<<"Stack Underflow!"<<std::endl;
        return;
    }
    Node<T>* current = head<T>;
    std::cout<<"Current Stack: ";
    while (current != nullptr){
        std::cout<<current->data<<" ";
        current = current->next;
    }
     std::cout<<std::endl;
}


```

STL Stack:

VSCODES > Stax > ILO C > STL > mainSTL.cpp > main()

```cpp
#include <iostream>
#include <string>
#include <stack>

std::string checkword;
char match;
std::stack<char> m;
int main(){
    std::cout<<"Input string: ";
    std::cin>>checkword;
    for (int i = 0; i < checkword.length(); i++){
        if (checkword[i] == '(' or checkword[i] == '[' or checkword[i] == '{' ){
            std::cout<<"Pushing: "<<checkword[i]<<std::endl;
            m.push(checkword[i]);
        }
        else if (checkword[i] == ')' || checkword[i] == '}' || checkword[i] == ']' ){
            if(m.empty()){
                std::cout<<"ERROR! STACK EMPTY!"<<std::endl;
                return 0;
            }

            if(m.top() == '(' && checkword[i] == ')' || m.top() == '{' && checkword[i]=='}' || m.top() == '[' && checkword[i] == ']' ){
             std::cout<<"Popping: "<<m.top()<<std::endl;
                m.pop();
        }

        }
    }
    if(!m.empty()){
        std::cout<<"Error: NOT VALID"<<std::endl;
    }
    else{
        std::cout<<"IS VALID"<<std::endl;
    }
    return 0;
}
```

Summary/Analysis:

All of the supplementary code all work the same. Same output, same system. The only difference is the concepts used to achieve that same goal. The array method makes use of an array, keeping track of the top element; only limited by the declared size of the programmer and being unable to deallocate that element from memory after popping. This is not the case for the Linked-List method as it allows for complete deletion or deallocation of the popped element. The STL method makes use of the stack library, with pre-written functions available for use. The sorting or pairing algorithm works the same for all the methods, looking for matches from the pushed opening symbols, popping when they do, ignoring irrelevant characters.

Concluding statement / Feedback: How well did you think you did in this activity? What are your areas for Improvement?

I would say that I did my best for the current knowledge and concepts taught to me. However, there is this underlying feeling that I could shorten or compact some of my code or an even easier way to sort and check for matches, other than that, I would say I did decent.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

COMPUTER ENGINEERING

QUEZON CITY

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.